

THE NODAL CODEBASE *explained.*

A visual study of how the interface, server, database, integrations, and safeguards connect.



READ THIS AS A SYSTEM, NOT A LIST OF FILES

- Start with what a member experiences.
- Follow the request behind the screen.
- Read the real code with plain-language notes.
- Finish with the security and study maps.

Contents

The chapter order follows one idea from the screen to the database and back. Code walkthroughs sit inside the chapters where they belong.

Reader's Toolkit	5
1. What NODAL Is Today	6
2. The Architecture in One Diagram	7
3. Repository Map	8
4. How a Request Travels Through NODAL	8
4.1 Static JavaScript, CSS, and images	9
4.2 HTML pages	9
4.3 API requests	9
5. The Vercel Runtime	10
api/index.js	11
vercel.json	11
scripts/build-static.js	11
6. Frontend: Pages and Responsibilities	12
web/pages/index.html	12
web/pages/login.html	12
web/pages/dashboard.html	12
web/pages/profile.html	12
web/pages/payments.html	12
7. Frontend: Key JavaScript	12
web/scripts/auth.js	14
web/scripts/dashboard.js	14
web/scripts/recs.js	15
web/scripts/payments.js	15
web/scripts/profile.js	15
web/scripts/app.js	15
web/scripts/i18n.js, nav.js, and script.js	15
8. Backend: The Central HTTP Router	16
Important helpers	18
API surface	19
9. The Repository Pattern	20
10. Supabase Integration	21
Two credential levels	23

Account creation	23
Session resolution and refresh	24
Profile persistence	24
11. SQLite Integration	25
12. Profile Policy and Domain Modeling	26
server/domain.js	26
server/profile-policy.js	27
13. Recommendation Engine	27
Input graph	28
Signals	29
Graph traversal	29
Collaborative filtering	29
Engagement decay	29
Final recommendation	29
Recommendation caching	30
14. External API Connections	30
14.1 Supabase Auth API	31
14.2 Supabase PostgREST API	31
14.3 Stripe API	31
14.4 GeoDB Cities API	32
14.5 Redis	32
14.6 Integrations not active yet	32
15. Supabase Database Schema	33
20260709_production_core.sql	33
20260710000000_security_hardening.sql	33
20260710010000_advisor_hardening.sql	33
20260710020000_rls_performance.sql	33
16. How Row Level Security Works	34
17. The Security Model	35
Threat: a secret appears in Git or browser code	37
Threat: an anonymous visitor opens private UI or APIs	37
Threat: one member reads or changes another member's data	37
Threat: a cross-site page submits a mutation using the member's cookies	38
Threat: session theft through browser JavaScript	38
Threat: malicious or oversized input	38
Threat: brute-force login or recommendation poisoning	38
Threat: forged or replayed Stripe events	39
Threat: accidental source or operational-file exposure	39
Browser hardening	39
18. Important Design Decisions and Their Tradeoffs	40

Keep the existing frontend stack 40

Put a server boundary in front of Supabase 40

Use Supabase in production and SQLite locally 40

Serve assets statically but HTML through Node 40

Make recommendations pure and data-source independent 40

Keep payments fail-closed 40

Use an explicit deployment allowlist 41

19. Reliability and Performance Work 41

20. Tests as Executable Documentation 42

21. What Changed Over the Life of the Repository 43

22. Current Limitations and Future Work 44

23. Recommended Reading Order 45

 Pass 1: understand the shape 45

 Pass 2: follow one authenticated request 45

 Pass 3: follow profile state 45

 Pass 4: follow recommendations 45

 Pass 5: study the security story 45

24. Practical Study Exercises 46

25. The Core Mental Model 46

Reader's Toolkit

You do not need to read JavaScript fluently. Understand the human story first, then use the line numbers and notes to recognize the symbols that control each behavior.

CODE LEGEND

`if` is a gate. `return` stops a function. `await` waits for outside work. `req` is the incoming request. `res` is the outgoing response. `//` begins a developer note.

TERM	MEANING IN NODAL
Function	A named set of instructions. Inputs go in; a result or action comes out.
Class	A blueprint for stateful objects. NODAL mostly uses functions; cache implementations are classes.
Object	A group of named values, like one member profile or one configuration.
API	A controlled doorway through which one piece of software asks another piece to act.
Endpoint	One specific API doorway, such as <code>POST /api/auth/login</code> .
Request	What the browser sends: method, path, headers, cookies, and sometimes a body.
Response	What the server returns: status, headers, cookies, and a body.
<code>async</code> / <code>await</code>	The code waits for network or database work without freezing every other request.
Database	Durable storage: Supabase Postgres in production and SQLite locally.
Migration	A versioned database change applied in a known order.
Cookie	A browser-held value sent to the same site; NODAL session tokens are <code>HttpOnly</code> .
Environment variable	Deployment configuration stored outside source code, especially secrets.
Cache	A temporary result copy that improves speed but is not the source of truth.
Test	Executable evidence that a behavior or boundary still works.

WHY THERE ARE FEW CLASSES

NODAL is function-oriented because routing, validation, scoring, and translation are easy to test as functions. Classes appear where an object owns changing internal state, such as `MemoryCache` and `RedisCache`.

DOCUMENT SCOPE

Snapshot: `main` at commit `e761de8` on 2026-07-10. This guide explains the repository as a system: what each area owns, how a request travels through it, why the main architectural decisions were made, how external APIs are connected, and how the security layers work together. It deliberately does not include real environment values or credentials.

1. What NODAL Is Today

THE PRODUCT IN ONE SENTENCE

People meet
through trusted
urban knowledge.

The code exists to make that
relationship real, private,
explainable, and durable.



NODAL is a lightweight full-stack community platform for people working on more just, sustainable, and human cities. The current code supports:

- Public landing and membership pages.
- Supabase email/password authentication in production.
- Secure server-managed sessions.
- Private dashboard, profile, and payments pages.
- Persisted member profiles and onboarding information.
- Consent-aware member discovery.
- A graph-based recommendation engine.
- Follow and interaction events that affect recommendations.
- Personal-data export and account deletion.
- Stripe Checkout and signed subscription webhooks when Stripe is configured.
- Global city autocomplete through a server-side GeoDB proxy.
- SQLite as a complete local and automated-test fallback.
- Optional Redis-backed recommendation caching.
- Vercel deployment with static asset delivery and a serverless Node request boundary.

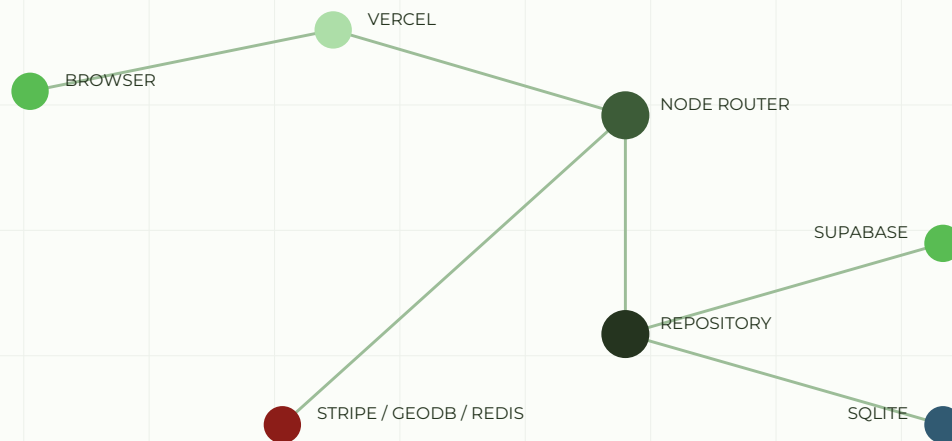
The most important architectural idea is this:

The browser renders the interface, but the server owns authentication, authorization, secrets, persistence, external-service calls, and sensitive decisions.

The app stayed on plain HTML, CSS, and JavaScript. Production readiness was added around that existing interface instead of replacing it with React, Next.js, or another framework.

2. The Architecture in One Diagram

SYSTEM MAP / REQUESTS MOVE LEFT TO RIGHT



The browser expresses intent. The server proves and authorizes it. The repository changes durable state.

There are four main boundaries:

- 1 Presentation boundary: `web/` contains everything the browser displays or executes.
- 2 HTTP and security boundary: `server/server.js` receives every HTML or API request that needs application logic.
- 3 Persistence boundary: `server/repository.js` selects Supabase or SQLite without changing the HTTP routes.
- 4 Deployment boundary: `api/index.js`, `vercel.json`, and `scripts/build-static.js` adapt the same Node app to Vercel.

3. Repository Map

Path	Responsibility
web/pages/	Source HTML pages. HTML is served by Node so private pages can be protected.
web/scripts/	Browser behavior, rendering, forms, and calls to the NODAL API.
web/styles/	Shared site styles and dashboard-specific styles.
web/assets/optimized/	WebP images that are allowed into production.
web/assets/source/	Editable PNG originals; not copied to production.
server/	HTTP routing, auth, validation, domain logic, persistence adapters, payments, cache, and recommendations.
api/index.js	Vercel serverless adapter for the Node server.
scripts/	Build, database migration, and local seed utilities. These are not runtime modules.
supabase/migrations/	Ordered production schema, RLS policies, indexes, views, triggers, and Stripe RPC.
tests/	Node's built-in test suite. The current suite contains 77 tests.
public/	Generated Vercel output. It is not source and should not be edited manually.
.github/workflows/ci.yml	Continuous integration checks.
vercel.json	Build, static delivery, headers, function packaging, and rewrites.
.env.example	Safe variable-name template without real credentials.
DEPLOYMENT.md	Operational setup and production checklist.

The repository was reorganized by responsibility while keeping every public URL stable. For example, the source file moved to `web/scripts/dashboard.js`, but the browser still requests `/dashboard.js`.

4. How a Request Travels Through NODAL

A REQUEST HAS FIVE CHECKPOINTS



Private pages are protected before HTML is sent

SOURCE / server/server.js / LINES 547-571

```

547 const server = http.createServer(async (req, res) => {
548   try {
549     const url = new URL(req.url, 'http://127.0.0.1');
550     const { pathname } = url;
551     const canonical = canonicalPathname(pathname);
552     const isApiRequest = pathname.startsWith('/api/');
553     const pageNeedsSession = !isApiRequest
554       && canonical
555       && (PRIVATE_PAGES.has(canonical) || canonical === '/login.html');
556     const session = useDb && (isApiRequest || pageNeedsSession)
557       ? await repository.resolveSession(req)
558       : { user: null, cookies: [] };
559     const sessionUser = session.user;
560     if (session.cookies?.length) res.setHeader('Set-Cookie', session.cookies);
561
562     if (!pathname.startsWith('/api/')) {
563       if (useDb && canonical && PRIVATE_PAGES.has(canonical) && !sessionUser) {
564         redirect(res, `/login.html?next=${encodeURIComponent(canonical)}`);
565         return;
566       }
567       if (useDb && canonical === '/login.html' && sessionUser) {
568         redirect(res, safeNext(url.searchParams.get('next')));
569         return;
570       }
571       await serveStatic(req, res, pathname);

```

IN PLAIN LANGUAGE

The server decides whether this request needs a session. Without a member, a private page becomes a login redirect and the function stops.

WHY THIS DECISION EXISTS

The private HTML never leaves the server until the session check succeeds.

4.1 Static JavaScript, CSS, and images

- 1 `npm run build` first runs the entire test suite.
- 2 `scripts/build-static.js` copies an explicit list of browser scripts, styles, and optimized WebP images to `public/`.
- 3 Vercel serves those files directly, with reusable cache headers from `vercel.json`.
- 4 Static asset requests do not need a database session and do not invoke the serverless function.

This improved loading speed and cost. It also created a security allowlist: only named browser files enter the public build.

4.2 HTML pages

HTML is intentionally not copied to `public/`. Requests for `/`, `/login.html`, `/dashboard.html`, `/profile.html`, and `/payments.html` pass through the Node server.

That lets `createApp()` in `server/server.js` enforce these rules before sending HTML:

- Anonymous users requesting a private page are redirected to `/login.html?next=...`
- Authenticated users requesting the login page are redirected to the safe destination.
- The `next` destination is normalized by `safeNext()` to prevent an open redirect.
- Only explicitly mapped pages can be read from disk.

4.3 API requests

All `/api/*` requests enter `createApp()` in `server/server.js`. The request flow is:

- 1 Parse the URL.

- 2 Resolve the session when the route needs it.
- 3 Propagate refreshed auth cookies if Supabase refreshed the token.
- 4 Match the HTTP method and route.
- 5 Enforce auth, same-origin rules, rate limits, and input validation.
- 6 Delegate persistence to the selected repository.
- 7 Return a deliberately small JSON response.
- 8 Hide unexpected internal errors behind `internal error` while logging a sanitized server-side message.

The central router begins at `createApp()` (server/server.js#L533).

5. The Vercel Runtime

The Node server is adapted to Vercel

SOURCE / `api/index.js` / LINES 1-19

```
1 import { createApp, validateRuntimeConfig } from '../server/server.js';
2 import { createCache } from '../server/cache.js';
3
4 let appPromise;
5
6 async function app() {
7   if (!appPromise) {
8     appPromise = (async () => {
9       validateRuntimeConfig();
10      return createApp({ cache: await createCache() });
11    })();
12   }
13   return appPromise;
14 }
15
16 export default async function handler(req, res) {
17   const server = await app();
18   server.emit('request', req, res);
19 }
```

IN PLAIN LANGUAGE

The first request validates configuration, creates the cache, and builds the application. Warm requests reuse it.

WHY THIS DECISION EXISTS

Local and serverless environments share one router instead of drifting into two implementations.

Only approved frontend assets enter production

SOURCE / scripts/build-static.js / LINES 7-39

```
7  const STATIC_SCRIPTS = [  
8    'app.js',  
9    'auth.js',  
10   'dashboard.js',  
11   'i18n.js',  
12   'nav.js',  
13   'payments.js',  
14   'profile.js',  
15   'recs.js',  
16   'script.js',  
17 ];  
18 const STATIC_STYLES = ['dashboard.css', 'styles.css'];  
19 const STATIC_ASSETS = [  
20   'latam-map.webp',  
21   'nodal-community.webp',  
22   'nodal-wordmark.webp',  
23 ];  
24  
25 await mkdir(OUTPUT, { recursive: true });  
26 await Promise.all(STATIC_SCRIPTS.map((file) => copyFile(  
27   path.join(WEB_ROOT, 'scripts', file),  
28   path.join(OUTPUT, file),  
29 )));  
30 await Promise.all(STATIC_STYLES.map((file) => copyFile(  
31   path.join(WEB_ROOT, 'styles', file),  
32   path.join(OUTPUT, file),  
33 )));  
34 await rm(path.join(OUTPUT, 'assets'), { recursive: true, force: true });  
35 await mkdir(path.join(OUTPUT, 'assets'), { recursive: true });  
36 await Promise.all(STATIC_ASSETS.map((file) => copyFile(  
37   path.join(WEB_ROOT, 'assets', 'optimized', file),  
38   path.join(OUTPUT, 'assets', file),  
39 )));
```

IN PLAIN LANGUAGE

The deployable scripts, styles, and images are named explicitly and copied into public/.

WHY THIS DECISION EXISTS

New internal files remain private unless someone intentionally adds them to this allowlist.

api/index.js

Vercel expects a serverless request handler, while the app was originally built around Node's `http.Server`. The adapter bridges those two models.

It lazily creates one application instance, validates production configuration, creates the cache once, and forwards Vercel's incoming request into the Node server. Warm serverless invocations can therefore reuse the initialized app and cache rather than rebuilding them on every request.

vercel.json

The Vercel configuration does four jobs:

- Uses `npm run build` and publishes `public/`.
- Includes `web/pages/**` inside the serverless function because HTML is not public static output.
- Adds cache and security headers to scripts, styles, and images.
- Rewrites unresolved routes to `api/index.js`.

The broad rewrite does not make repository files public. The server still applies its explicit path allowlist.

scripts/build-static.js

The build script lists every deployable browser file by name. This was chosen over "copy the whole frontend directory" because an explicit list makes accidental publication much harder. Source PNGs, HTML, tests, internal docs, SQL, local data, and server code are never copied into `public/`.

6. Frontend: Pages and Responsibilities

web/pages/index.html

The public NODAL entry point. It contains the product story, program/community/resource tabs, the network visual, and membership entry points. Its behavior comes mainly from `app.js`, `script.js`, `nav.js`, and `i18n.js`.

web/pages/login.html

Contains both sign-in and account-creation forms. The forms submit to NODAL's own API rather than calling Supabase directly.

web/pages/dashboard.html

The main authenticated workspace. It renders the member's profile state, discovery surfaces, requests, matching, notification state, and account controls.

web/pages/profile.html

The authenticated profile view. It binds to the active member returned by `/api/auth/me` rather than a hardcoded demo identity.

web/pages/payments.html

Displays membership pricing and billing status. Public labels come from server configuration, while checkout creation remains server-side.

7. Frontend: Key JavaScript

The login form calls NODAL, not Supabase

SOURCE / `web/scripts/auth.js` / LINES 32-61

```
32  async function post(path, body) {
33    const res = await fetch(path, {
34      method: 'POST',
35      headers: { 'Content-Type': 'application/json' },
36      body: JSON.stringify(body),
37    });
38    const data = await res.json().catch(() => ({}));
39    if (!res.ok) throw new Error(data.error || 'Request failed');
40    return data;
41  }
42
43  const loginForm = $('#loginForm');
44  const signupForm = $('#signupForm');
45
46  loginForm?.addEventListener('submit', async (e) => {
47    e.preventDefault();
48    const error = $('#loginError');
49    clearError(error);
50    const submit = loginForm.querySelector('button[type="submit"]');
51    submit.disabled = true;
52    try {
53      await post('/api/auth/login', {
54        email: $('#loginEmail').value.trim(),
55        password: $('#loginPassword').value,
56      });
57      location.assign(safeNext);
58    } catch (err) {
59      setError(error, err.message);
60      submit.disabled = false;
61    }
62  });
```

IN PLAIN LANGUAGE

The form converts email and password into JSON and posts them to NODAL's login endpoint.

WHY THIS DECISION EXISTS

Credentials pass through NODAL's validation and rate limit; the browser needs no Supabase secret.

One frontend helper handles API failures

SOURCE /web/scripts/dashboard.js / LINES 13-25

```
13  async function api(path, options = {}) {
14    const res = await fetch(path, {
15      ...options,
16      headers: { 'Content-Type': 'application/json', ...(options.headers || {}) },
17    });
18    if (res.status === 401) {
19      location.assign(`/login.html?next=${encodeURIComponent(location.pathname)}`);
20      throw new Error('authentication required');
21    }
22    const data = await res.json().catch(() => ({}));
23    if (!res.ok) throw new Error(data.error || `api ${res.status}`);
24    return data;
25  }
```

IN PLAIN LANGUAGE

Every dashboard request uses this helper. It attaches JSON headers, handles 401, parses JSON, and normalizes errors.

WHY THIS DECISION EXISTS

Central response behavior keeps every dashboard feature consistent.

Profile state becomes a controlled payload

SOURCE /web/scripts/dashboard.js / LINES 47-62

```
47  function serializeUser(user) {
48    return {
49      fullName: user.name,
50      title: user.role,
51      city: user.city,
52      interests: user.topics.map((t) => String(t.name || t.toLowerCase()).filter(Boolean),
53      active: user.active,
54      topics: user.topics,
55      skills: user.skills,
56      indicators: user.indicators,
57      partC: user.partC,
58      requests: user.requests,
59      mentorApplied: user.mentorApplied,
60      assessed: user.assessed,
61      notifRead: state.notifRead,
62    };
63  }
```

IN PLAIN LANGUAGE

serializeUser selects which profile fields the browser asks to save.

WHY THIS DECISION EXISTS

This minimizes data at the client boundary; the server still validates every value.

The interface trusts the server's canonical answer

SOURCE /web/scripts/dashboard.js / LINES 91-105

```

91  function setUser(user, persist = true) {
92    U = user;
93    if (persist) { state.user = user; touchUser(); }
94    applyAll();
95  }
96  async function saveProfile() {
97    if (!U) return;
98    const data = await api('/api/me', { method: 'PATCH', body:
99      JSON.stringify(serializeUser(U)) });
100    U = normalizeApiUser(data.user);
101    state.user = U;
102    state.notifRead = U.notifRead;
103  }
104  function touchUser() {
105    if (state.user) state.user = U;
106    saveProfile().catch(() => {});

```

IN PLAIN LANGUAGE

After PATCH /api/me, local state is replaced by the normalized user returned by the server.

WHY THIS DECISION EXISTS

Cleaned values, defaults, and server-owned decisions immediately become the displayed truth.

web/scripts/auth.js

This file owns sign-up and sign-in form behavior.

Key logic:

- Mirrors the server's safe local return-path rule.
- Sends credentials with POST /api/auth/signup or POST /api/auth/login.
- Handles the "confirmation email required" response separately from an active session.
- Redirects only after the server confirms success.
- Does not keep passwords or tokens in browser storage.

The browser receives an HttpOnly cookie through the response, but JavaScript cannot read it. Future requests send it automatically because it belongs to the same origin.

web/scripts/dashboard.js

This is the largest frontend controller and is the best file for studying the UI-to-API contract.

Important functions and flows:

- `api()` is the shared fetch wrapper. A 401 sends the user back to login.
- `normalizeApiUser()` converts the server model into the shape expected by the UI.
- `serializeUser()` creates the allowed profile update payload.
- `saveProfile()` sends PATCH /api/me and replaces in-memory state with the server's canonical response.
- The startup path calls GET /api/auth/me; the page does not invent an authenticated user locally.
- City autocomplete debounces requests to /api/cities, cancels stale requests with `AbortController`, and renders returned city labels.
- Directory search calls /api/users, which contains only consented members.
- Data export calls /api/me/export.

- Account deletion calls **DELETE /api/me** and requires the member to type their email.
- Logout calls **/api/auth/logout** and lets the server revoke/clear the session.

The dashboard keeps current display state in memory while the page is open. Persistence happens only after an API response succeeds.

One honest limitation: some Projects, Knowledge, and Opportunities content is still editorial/static interface content. Member identity, profile data, directory membership, recommendations, follows, interactions, auth, and billing state are server-backed; not every dashboard card has a production content API yet.

web/scripts/recs.js

This file owns the recommendation deck:

- 1 Check **/api/auth/state**.
- 2 Load **/api/recommendations/me**.
- 3 Render ranked candidates and explanation signals.
- 4 Send a follow or skip interaction.
- 5 Refresh the deck after the graph changes.

The client never computes trust or matching scores. It only renders server results.

web/scripts/payments.js

This file:

- Loads public billing labels from **/api/billing/config**.
- Loads the authenticated member's subscription state from **/api/billing/status**.
- Posts the chosen billing cycle to **/api/checkout**.
- Navigates to the Stripe-hosted checkout URL returned by the server.

It never receives the Stripe secret key and never handles card information.

web/scripts/profile.js

Loads the current member and consented directory data, normalizes that model, and renders the active profile rather than a fixed sample person.

web/scripts/app.js

Controls the public landing experience, including tabs, graph interactions, and public billing labels. The amount shown in the interface comes from the server, not a duplicated frontend constant.

web/scripts/i18n.js, nav.js, and script.js

- **i18n.js** owns English, Portuguese, and Spanish strings and stores only the language preference in `localStorage`.
- **nav.js** owns navigation behavior and responsive menus.
- **script.js** contains shared page interactions and visual behavior.

The decision to keep plain JavaScript preserved the visual system, avoided a risky framework migration, kept the build small, and made browser debugging direct. The tradeoff is that **dashboard.js** now has several responsibilities and would be the first frontend file to modularize if the interface grows

substantially.

8. Backend: The Central HTTP Router

Signup is an ordered series of gates

SOURCE / server/server.js / LINES 597-631

```

597     if (useDb && req.method === 'POST' && pathname === '/api/auth/signup') {
598       if (!sameOrigin(req)) { send(res, 403, { error: 'cross-origin request rejected' });
        return; }
599       const rate = authLimiter.take(`signup:${clientIp(req)}`);
600       if (!rate.ok) {
601         send(res, 429, { error: 'too many authentication attempts' }, { 'Retry-After':
          String(rate.retryAfter) });
602         return;
603       }
604       const body = await readJsonBody(req);
605       const fullName = String(body.fullName ?? body.name ?? '').trim();
606       const email = String(body.email ?? '').trim().toLowerCase();
607       const password = String(body.password ?? '');
608       if (fullName.length < 2) { send(res, 400, { error: 'full name is required' });
        return; }
609       if (!validateEmail(email)) { send(res, 400, { error: 'valid email is required' });
        return; }
610       if (!validatePassword(password)) { send(res, 400, { error: 'password must be at
        least 8 characters' }); return; }
611       let result;
612       try {
613         result = await repository.signup({ fullName, email, password, env: process.env });
614       } catch (err) {
615         if (err?.status === 429) {
616           send(res, 429, { error: 'Confirmation email is temporarily unavailable. Please
            try again later.' });
617           return;
618         }
619         if (Number.isInteger(err?.status) && err.status >= 400) {
620           send(res, err.status < 500 ? err.status : 502, {
621             error: 'Account creation could not be completed. If you already confirmed your
              email, sign in.',
622           });
623           return;
624         }
625         throw err;
626       }
627       if (result.error) { send(res, result.status, { error: result.error }); return; }
628       send(res, result.status, {
629         user: result.user,
630         requiresEmailConfirmation: Boolean(result.requiresEmailConfirmation),
631       }, result.cookies?.length ? { 'Set-Cookie': result.cookies } : {});

```

IN PLAIN LANGUAGE

Origin, rate, body, name, email, and password checks happen before the repository creates an account.

WHY THIS DECISION EXISTS

Cheap safety checks run before expensive or sensitive work, and provider errors are translated safely.

Profile writes target only the signed-in member

SOURCE / server/server.js / LINES 691-697

```
691     if (useDb && req.method === 'PATCH' && pathname === '/api/me') {
692       if (!requireAuth(res, sessionUser)) return;
693       if (!sameOrigin(req)) { send(res, 403, { error: 'cross-origin request rejected' });
        return; }
694       const patch = sanitizeProfilePatch(await readJsonBody(req));
695       const user = await repository.updateUserProfile(sessionUser.id, patch);
696       send(res, 200, { user: repository.toApiUser(user) });
697       return;
```

IN PLAIN LANGUAGE

The route requires a session, checks origin, cleans JSON, and writes using sessionUser.id.

WHY THIS DECISION EXISTS

There is no browser-supplied user ID to manipulate into editing another profile.

Recommendation cache keys include graph state

SOURCE / server/server.js / LINES 715-735

```
715     let m = pathname.match(/^\/api\/recommendations\/([\/]+)$/);
716     if (m && req.method === 'GET') {
717       if (useDb && !requireAuth(res, sessionUser)) return;
718       const userId = resolveUserId(m[1], sessionUser, useDb);
719       if (!userId) { send(res, 403, { error: 'forbidden user scope' }); return; }
720       if (!ID_RE.test(userId)) { send(res, 400, { error: 'invalid user id' }); return; }
721       const activeStore = useDb ? await repository.loadGraphStore({ viewerId: userId }) :
        store;
722       if (!activeStore.users.has(userId)) { send(res, 404, { error: 'unknown user' });
        return; }
723       const key = cacheKey(userId, activeStore);
724       const cached = await cache.get(key);
725       if (cached) {
726         send(res, 200, cached, { 'X-Cache': 'HIT', 'Content-Type': 'application/json;
        charset=utf-8' });
727         return;
728       }
729     }
730     const recommendations = recommend(activeStore, userId) ?? [];
731     const payload = { userId, generatedAt: new Date().toISOString(), recommendations };
732     await cache.set(key, JSON.stringify(payload), REC_TTL_MS);
733     send(res, 200, payload, { 'X-Cache': 'MISS' });
734     return;
```

IN PLAIN LANGUAGE

The route resolves the current member, loads the visible graph, checks a fingerprinted cache key, and computes on a miss.

WHY THIS DECISION EXISTS

The cache accelerates reads but never becomes the source of truth.

The key backend file is `server/server.js`.

Important helpers

Function	Purpose
<code>securityHeaders()</code>	Adds baseline browser security headers.
<code>contentSecurityPolicy()</code>	Builds the CSP and limits where scripts, styles, images, connections, frames, and forms may go.
<code>safeNext()</code>	Accepts only safe local redirect destinations.
<code>canonicalPathname()</code>	Normalizes paths and rejects malformed encodings, null bytes, and traversal-like forms.
<code>createWindowRateLimiter()</code>	Implements bounded per-window limits for auth and interaction routes.
<code>publicBaseUrl()</code>	Selects a trusted deployment URL instead of trusting the request's <code>Host</code> header.
<code>createCitySearch()</code>	Validates, paces, caches, and proxies city searches.
<code>validateRuntimeConfig()</code>	Fails closed when a production deploy is missing required database, URL, cookie, or payment configuration.
<code>sameOrigin()</code>	Rejects cross-site state-changing requests.
<code>readJsonBody()</code>	Requires JSON and limits bodies to 32 KB.
<code>readRawBody()</code>	Limits raw webhook bodies to 128 KB.
<code>recordStripeEvent()</code>	Translates signed Stripe events into the database's atomic subscription mutation.
<code>graphFingerprint()</code>	Creates a stable recommendation-cache identity from the current graph state.
<code>sanitizeProfilePatch()</code>	Accepts and normalizes only supported profile fields.
<code>requireAuth()</code>	Returns 401 before protected logic runs.
<code>resolveUserId()</code>	Prevents a member from operating as another user. <code>me</code> resolves to the session owner.
<code>staticSourcePath()</code>	Maps public URLs to explicitly allowed source files.
<code>createApp()</code>	Wires the complete HTTP application.

API surface

Method and route	Responsibility	Main protections
GET /api/health	Lightweight health check.	No sensitive output.
GET /api/billing/config	Public labels and payment availability.	Returns no secrets.
GET /api/cities?q=	City autocomplete proxy.	Auth, validation, pacing, cache, provider failure fallback.
GET /api/auth/state	Boolean session state.	No profile data.
POST /api/auth/signup	Create Supabase/SQLite account.	Same-origin, rate limit, email/password/name validation, sanitized provider errors.
POST /api/auth/login	Create a session.	Same-origin, rate limit before password work, generic invalid-credential response.
POST /api/auth/logout	Revoke and clear session.	Same-origin.
GET /api/auth/me	Current canonical member model.	Auth required.
PATCH /api/me	Update the current profile.	Auth, same-origin, allowlisted patch, server-owned eligibility fields.
GET /api/me/export	Export the member's NODAL data.	Auth and session ownership.
DELETE /api/me	Delete the account and related data.	Auth, same-origin, typed email confirmation.
GET /api/users	Consented member directory.	Auth and consent filter.
GET /api/recommendations/me	Ranked member recommendations.	Auth, self scope, consent-aware graph, cache.
POST /api/users/me/follow	Follow a visible member.	Auth, same-origin, target validation, cache invalidation.
POST /api/users/me/interactions	Record bounded view/skip/like/message signals.	Auth, same-origin, type validation, rate limiting, bounded history.
GET /api/billing/status	Current member's subscription metadata.	Auth and user scope.
POST /api/checkout	Create Stripe Checkout Session.	Auth, same-origin, plan/cycle validation, trusted return URL, fail-closed config.
POST /api/stripe/webhook	Apply Stripe subscription events.	Raw-body limit, HMAC signature, timestamp tolerance, event ordering, atomic DB function.

9. The Repository Pattern

One function selects the persistence backend

SOURCE / server/repository.js / LINES 95-100

```
95 export function createRepository({ db, store, env = process.env, fetchImpl = fetch } = {}) {
96   if (db) return createSqliteRepository(db, { ownsDb: false });
97   if (store) return null;
98   if (dataBackend(env) === 'supabase') return createSupabaseRepository({ env, fetchImpl });
99   return createSqliteRepository(createDatabase(), { ownsDb: true });
100 }
```

IN PLAIN LANGUAGE

Tests may inject data. Production selects Supabase. Local runs create SQLite.

WHY THIS DECISION EXISTS

Frontend and route code keep one contract across every environment.

SQLite implements the same account operations

SOURCE / server/repository.js / LINES 28-52

```
28 function createSqliteRepository(db, { ownsDb = false } = {}) {
29   return {
30     kind: 'sqlite',
31     database: db,
32     cleanupExpiredSessions() {
33       deleteExpiredSessions(db);
34     },
35     async resolveSession(req) {
36       return { user: getSessionUser(db, req), cookies: [] };
37     },
38     async signup({ fullName, email, password, env = process.env }) {
39       if (getUserByEmail(db, email)) {
40         return { status: 409, error: 'signup could not be completed', cookies: [] };
41       }
42       const user = createUser(db, { fullName, email, passwordHash: await
hashPassword(password) });
43       const session = createSession(db, user.id, { env });
44       return { status: 201, user: toApiUser(user), cookies: [session.cookie] };
45     },
46     async login({ email, password, env = process.env }) {
47       const user = getUserByEmail(db, email);
48       if (!user || user.account_status !== 'active' || !(await verifyPassword(password,
user.password_hash))) {
49         return { status: 401, error: 'invalid email or password', cookies: [] };
50       }
51       const session = createSession(db, user.id, { env });
52       return { status: 200, user: toApiUser(user), cookies: [session.cookie] };
```

IN PLAIN LANGUAGE

The local repository exposes session resolution, signup, and login in the same shape as Supabase.

WHY THIS DECISION EXISTS

Local development exercises real behavior rather than a fabricated frontend identity.

server/repository.js prevents the HTTP router from knowing database-specific details.

createRepository() chooses:

- An injected database/store during focused tests.
- Supabase when DATA_BACKEND=supabase or the Vercel production environment selects it.
- SQLite for local development and most automated tests.

Both implementations expose the same conceptual operations: resolve session, sign up, log in, update profile, list directory members, load the graph, follow, record interactions, export/delete data, and apply billing events.

Why this decision matters:

- The frontend has one API contract in every environment.

- The recommendation engine receives the same graph shape from either database.
- Local development works without a cloud dependency.
- Automated tests are fast and deterministic.
- Production still uses managed Postgres, Supabase Auth, and RLS.
- Migrating persistence does not require rewriting the UI or router.

10. Supabase Integration

Public and secret Supabase capabilities stay separate

SOURCE / server/supabase.js / LINES 140-168

```
140 export function createSupabaseClients({ env = process.env, fetchImpl = fetch } = {}) {
141   const resolved = resolveSupabaseEnv(env, { requireServer: true });
142   return {
143     env: resolved,
144     publicConfig: { url: resolved.url, publishableKey: resolved.publishableKey },
145     browser: createRestClient({ url: resolved.url, key: resolved.publishableKey, fetchImpl
146   }),
147     admin: createRestClient({ url: resolved.url, key: resolved.serverKey, fetchImpl }),
148     user(accessToken) {
149       return createRestClient({
150         url: resolved.url,
151         key: resolved.publishableKey,
152         authToken: accessToken,
153         fetchImpl,
154       });
155     },
156   };
157 }
158 function refreshCookie(value, { env = process.env, maxAge = 60 * 60 * 24 * 30 } = {}) {
159   const secure = env.COOKIE_SECURE === 'true' || env.NODE_ENV === 'production' ? 'Secure'
160   : '';
161   return `${REFRESH_COOKIE}=${encodeURIComponent(value)}; HttpOnly; Path=/; Max-
162   Age=${maxAge}; SameSite=Lax${secure}`;
163 }
164 function sessionCookies(session, env) {
165   if (!session?.access_token) return [];
166   const accessMaxAge = Math.max(60, Number(session.expires_in) || 60 * 60);
167   const cookies = [sessionCookie(session.access_token, { env, maxAge: accessMaxAge })];
168   if (session.refresh_token) cookies.push(refreshCookie(session.refresh_token, { env }));
169   return cookies;
170 }
```

IN PLAIN LANGUAGE

A publishable-key client handles Auth; a secret-key client performs trusted server data work. Cookies are HttpOnly.

WHY THIS DECISION EXISTS

Even the lower-privilege client lives on the server. Page JavaScript receives neither keys nor tokens.

Expired access tokens refresh without page JavaScript

SOURCE / server/supabase.js / LINES 396-423

```
396   async resolveSession(req) {
397     const cookies = parseCookies(req.headers.cookie);
398     const accessToken = cookies.get(ACCESS_COOKIE);
399     const refreshToken = cookies.get(REFRESH_COOKIE);
400     if (accessToken) {
401       try {
402         const authUser = await authUserFromToken(accessToken);
403         if (!authUser?.id) return { user: null, cookies: clearSessionCookies(env) };
404         return { user: await existingOrEnsureProfile(authUser), cookies: [] };
405       } catch (err) {
406         if (!refreshToken || ![401, 403].includes(err?.status)) {
407           return { user: null, cookies: err?.status === 401 ? clearSessionCookies(env) :
408             [] };
409         }
410       }
411       if (!refreshToken) return { user: null, cookies: [] };
412       try {
413         const session = await browser.auth('/token', {
414           method: 'POST',
415           query: { grant_type: 'refresh_token' },
416           body: { refresh_token: refreshToken },
417         });
418         const authUser = session?.user || await authUserFromToken(session?.access_token);
419         if (!authUser?.id) return { user: null, cookies: clearSessionCookies(env) };
420         return { user: await existingOrEnsureProfile(authUser), cookies:
421           sessionCookies(session, env) };
422       } catch {
423         return { user: null, cookies: clearSessionCookies(env) };
424       }
425     }
```

IN PLAIN LANGUAGE

The repository tries the access token, then uses the refresh cookie when Supabase reports expiry.

WHY THIS DECISION EXISTS

The member stays signed in while sensitive tokens remain unavailable to injected scripts.

One UI profile is split across purpose-specific tables

SOURCE / server/supabase.js / LINES 350-390

```

350   await admin.rest('profiles', {
351     method: 'PATCH',
352     query: { id: `eq.${userId}` },
353     headers: { Prefer: 'return=representation' },
354     body: {
355       full_name: fullName,
356       preferred_name: fullName.split(/\s+/)[0] || fullName,
357       city_region: city,
358       bio: cleanString(partC.bio, 2000),
359       public_role: title,
360       updated_at: nowIso(),
361     },
362   });
363   await admin.rest('profile_preferences', {
364     method: 'POST',
365     query: { on_conflict: 'user_id' },
366     headers: { Prefer: 'resolution=merge-duplicates,return=minimal' },
367     body: [{
368       user_id: userId,
369       visibility: { directory: Boolean(partC.consent) },
370       notification_preferences: { dashboardRead: Boolean(rawAnswers.notifRead) },
371       data_consent: { directoryPublic: Boolean(partC.consent) },
372       updated_at: nowIso(),
373     }],
374   });
375   await admin.rest('onboarding_responses', {
376     method: 'POST',
377     query: { on_conflict: 'user_id' },
378     headers: { Prefer: 'resolution=merge-duplicates,return=minimal' },
379     body: [{
380       user_id: userId,
381       interests: 'interests' in patch ? asArray(patch.interests).map(String).slice(0, 12)
382       : current.interests,
383       skills: 'skills' in patch ? asArray(patch.skills).map(String).slice(0, 12) :
384       current.skills,
385       goals: 'goals' in patch ? asArray(patch.goals).map(String).slice(0, 12) :
386       current.goals,
387       contribution_preferences: rawAnswers.active,
388       availability: cleanString(partC.availability, 120),
389       mentoring_interest: rawAnswers.mentorApplied ? 'applied' : 'none',
390       raw_answers: rawAnswers,
391       updated_at: nowIso(),
392     }],
393   });

```

IN PLAIN LANGUAGE

Identity, consent, notifications, interests, skills, and goals are written to tables that match their purpose.

WHY THIS DECISION EXISTS

Privacy policy, RLS, export, and retention become easier to reason about.

The production adapter lives in `server/supabase.js`.

Two credential levels

`createSupabaseClients()` creates two server-side REST wrappers:

- A publishable-key Auth client for sign-up, password login, token refresh, user lookup, and logout.
- A secret-key admin client for trusted server-side PostgREST operations and admin account deletion.

The variable name `browser` inside this server module describes the low-privilege Supabase client type. It is still instantiated and used on the Node server. The website's browser JavaScript does not receive either Supabase key from NODAL.

`resolveSupabaseEnv()` validates the project URL and rejects a secret-looking key in the public-key slot or a publishable-looking key in the secret slot. This guards against a dangerous environment-variable swap.

Account creation

`signup()` calls Supabase Auth with the member's email, password, and name metadata. Supabase may return either:

- A session immediately, when email confirmation is disabled; or
- A user without an access token, when confirmation is required.

The code represents the second case as `202` plus `requiresEmailConfirmation: true`. It also handles repeated signup responses that intentionally obscure whether an email already exists.

`ensureProfile()` creates the matching `profiles` and `profile_preferences` records using conflict-safe inserts. This is why a confirmed Auth user and the application's profile records stay linked by the same UUID.

Session resolution and refresh

`resolveSession()` reads the HttpOnly access and refresh cookies:

- 1 Validate the access token through Supabase Auth.
- 2 Load or repair the member's profile bundle.
- 3 If the access token expired, exchange the refresh token for a new session.
- 4 Return replacement cookies to the router.
- 5 Clear cookies if the session is no longer valid.

No access or refresh token is stored in `localStorage` or exposed to page JavaScript.

Profile persistence

The public UI thinks of the member as one object. The database separates that object by data purpose:

- `profiles`: identity and public-facing profile fields.
- `profile_preferences`: visibility, notification settings, and consent.
- `onboarding_responses`: interests, skills, goals, activity, availability, and structured answers.

`bundleForUser()` loads those records in parallel. `toApiUserFromSupabase()` combines them into the stable frontend model.

`upsertProfileState()` performs the reverse operation. It validates the incoming patch, preserves fields that were not changed, writes each table, and reloads the canonical result.

Crucially, client input cannot directly forge assessment markers, trust indicators, or mentor eligibility. `profile-policy.js` calculates and cleans those fields server-side.

11. SQLite Integration

Local passwords use scrypt and timing-safe comparison

SOURCE / server/auth.js / LINES 15-39

```

15 export function validateEmail(email) {
16   return typeof email === 'string' && /^[^s@]+@[^s@]+\.[^s@]+$/.test(email.trim());
17 }
18
19 export function validatePassword(password) {
20   return typeof password === 'string' && password.length >= 8 && password.length <= 160;
21 }
22
23 export async function hashPassword(password) {
24   const salt = randomBytes(16);
25   const hash = await scrypt(password, salt, 64, SCRYPT);
26   return `scrypt${SCRYPT.N}${SCRYPT.r}${SCRYPT.p}${b64(salt)}${b64(hash)}`;
27 }
28
29 export async function verifyPassword(password, encoded) {
30   const parts = String(encoded || '').split('$');
31   if (parts.length !== 6 || parts[0] !== 'scrypt') return false;
32   const [, n, r, p, saltText, hashText] = parts;
33   const expected = fromB64(hashText);
34   const actual = await scrypt(password, fromB64(saltText), expected.length, {
35     N: Number(n),
36     r: Number(r),
37     p: Number(p),
38     maxmem: 64 * 1024 * 1024,
39   });

```

IN PLAIN LANGUAGE

Signup derives a salted scrypt hash. Login derives the same form and compares bytes safely.

WHY THIS DECISION EXISTS

The original password is never stored in the local database.

The local session token is protected twice

SOURCE / server/auth.js / LINES 57-80

```

57 export function tokenHash(token) {
58   return createHash('sha256').update(token).digest('base64url');
59 }
60
61 function cookieSecure(env = process.env) {
62   return env.COOKIE_SECURE === 'true' || env.NODE_ENV === 'production';
63 }
64
65 export function sessionCookie(token, { env = process.env, maxAge = SESSION_DAYS * 24 * 60 *
66   60 } = {}) {
67   const secure = cookieSecure(env) ? 'Secure' : '';
68   return `${COOKIE}=${encodeURIComponent(token)}; HttpOnly; Path=/; Max-Age=${maxAge};
69   SameSite=Lax${secure}`;
70 }
71
72 export function clearSessionCookie(env = process.env) {
73   return sessionCookie('', { env, maxAge: 0 });
74 }
75
76 export function createSession(db, userId, { env = process.env } = {}) {
77   const token = b64(randomBytes(32));
78   const expiresAt = new Date(Date.now() + SESSION_DAYS * 24 * 60 * 60 * 1000).toISOString();
79   db.prepare('INSERT INTO sessions (id, user_id, token_hash, expires_at) VALUES (?, ?, ?,
80     ?)')
81     .run(randomUUID(), userId, tokenHash(token), expiresAt);
82   return { token, cookie: sessionCookie(token, { env }) };

```

IN PLAIN LANGUAGE

A random token goes to an HttpOnly cookie; only its SHA-256 hash is stored in SQLite.

WHY THIS DECISION EXISTS

A database leak alone does not reveal the active browser token.

`server/db.js` is the local persistence implementation. It uses Node's SQLite support and owns:

- Local schema migrations.

- User creation and profile updates.
- Directory consent filtering.
- Follow and interaction persistence.
- Session records.
- Subscription metadata and ordered Stripe-event application.
- Personal-data export.
- Cascade-style account deletion.

`server/auth.js` supplies the local auth implementation:

- Email and password validation.
- Script password hashing.
- Timing-safe password verification.
- Random session tokens.
- SHA-256 token hashes in the database rather than raw session tokens.
- HttpOnly, SameSite, and production Secure cookies.

SQLite is not a fake frontend mode. It is a functional local backend with the same route contract. Production uses Supabase because serverless instances cannot rely on a durable local SQLite file.

12. Profile Policy and Domain Modeling

Mentor eligibility is a server-owned rule

SOURCE / `server/profile-policy.js` / LINES 56-76

```

56 export function cleanRequests(value, currentRequests = {}, nextPartC = DEFAULT_PART_C) {
57   const incoming = value && typeof value === 'object' ? value : {};
58   const out = {};
59   for (const key of REQUEST_KEYS) {
60     const alreadyRequested = currentRequests[key] === true;
61     const requested = incoming[key] === true;
62     if (key === 'project') {
63       const hasDepthProfile = ['bio', 'linkedin', 'portfolio', 'references', 'availability']
64         .every((field) => String(nextPartC[field] ?? '').trim() !== '');
65       out[key] = alreadyRequested || (requested && hasDepthProfile);
66     } else {
67       out[key] = alreadyRequested || requested;
68     }
69   }
70   return out;
71 }
72
73 export function canApplyForMentor({ assessed, topics, indicators }) {
74   const maxTopicLevel = Math.max(0, ...topics.map((topic) => Number(topic.level) || 0));
75   return Boolean(assessed) && maxTopicLevel >= 3 && indicators.transmission !== 'No';
76 }

```

IN PLAIN LANGUAGE

Requests are cleaned by category. Eligibility requires assessment, topic level, and trusted indicators.

WHY THIS DECISION EXISTS

A browser may request mentor status but cannot manufacture the evidence that permits it.

`server/domain.js`

Defines the canonical default profile/preference structures used when an account is initialized.

server/profile-policy.js

Centralizes rules that must not depend on what the browser claims:

- Normalize profile section C.
- Restrict indicator values.
- Clean interests/topics and requests.
- Preserve existing server-owned markers.
- Decide whether the member can apply as a mentor.

This separation is a security and maintainability decision. UI rendering can change without silently changing eligibility rules, and profile writes use one policy in both Supabase and SQLite modes.

13. Recommendation Engine

The scoring policy is visible and testable

SOURCE / server/engine.js / LINES 11-24

```
11 /* NOTE: the "How matching works" tab on the landing page displays these
12    weights - keep index.html's sig-bars in sync when changing them */
13 export const WEIGHTS = { interests: 0.30, mutuals: 0.20, engagement: 0.20, activity: 0.10,
    city: 0.10, complement: 0.10 };
14 export const DECAY = 0.5;          // score multiplier per extra hop
15 export const MAX_DEPTH = 3;
16 export const LINKEDIN_BOOST = 1.05; // verifiability prior on the final score
17 const MIX = { traversal: 0.45, direct: 0.30, cf: 0.25 };
18
19 /* complementary disciplines - pairs that historically need each other on projects */
20 const COMPLEMENTS = [
21   ['research', 'engineer'], ['research', 'technolog'], ['research', 'designer'],
22   ['planner', 'community'], ['planner', 'economist'], ['planner', 'anthropolog'],
23   ['architect', 'community'], ['engineer', 'community'],
24 ];
```

IN PLAIN LANGUAGE

Six direct signals sum to one, with separate constants for graph depth, decay, and final mixing.

WHY THIS DECISION EXISTS

The ranking assumptions are inspectable rather than hidden inside an external model.

Direct affinity combines six normalized signals

SOURCE / server/engine.js / LINES 75-86

```
75 export function edgeWeight(store, adj, a, b) {
76   const ua = store.users.get(a), ub = store.users.get(b);
77   const engagement = getEngagement(store, a, b);
78   return (
79     WEIGHTS.interests * jaccard(ua.interests, ub.interests) +
80     WEIGHTS.mutuals * mutualScore(adj, a, b) +
81     WEIGHTS.engagement * (engagement / (engagement + 3)) + // saturating
82     WEIGHTS.activity * jaccard(ua.active, ub.active) +
83     WEIGHTS.city * cityScore(ua, ub) +
84     WEIGHTS.complement * complementScore(ua, ub)
85   );
86 }
```

IN PLAIN LANGUAGE

edgeWeight multiplies each signal by its policy weight and adds the results. Engagement saturates.

WHY THIS DECISION EXISTS

Every candidate can be explained using recognizable evidence.

Three ranking perspectives become one score

SOURCE / server/engine.js / LINES 136-158

```
136 export function recommend(store, userId, { limit = 6 } = {}) {
137   const me = store.users.get(userId);
138   if (!me) return null;
139   const adj = buildAdjacency(store);
140   const following = store.follows.get(userId);
141
142   const traversal = normalize(traversalScores(store, adj, userId));
143   const cf = normalize(cfScores(store, userId));
144   const direct = normalize(new Map(
145     [...store.users.keys()]
146       .filter((id) => id !== userId)
147       .map((id) => [id, edgeWeight(store, adj, userId, id)]),
148   ));
149
150   const results = [];
151   for (const [id, user] of store.users) {
152     if (id === userId || following.has(id)) continue;
153     let score =
154       MIX.traversal * (traversal.get(id) ?? 0) +
155       MIX.direct * (direct.get(id) ?? 0) +
156       MIX.cf * (cf.get(id) ?? 0);
157     if (score <= 0) continue;
158     if (user.linkedin) score *= LINKEDIN_BOOST;
```

IN PLAIN LANGUAGE

Graph traversal, collaborative filtering, and direct affinity are normalized and blended after exclusions.

WHY THIS DECISION EXISTS

Network proximity, profile fit, and similar members all contribute without one signal deciding everything.

Recommendations include human-readable reasons

SOURCE / server/engine.js / LINES 160-184

```
160   const shared = me.interests.filter((i) => user.interests.includes(i));
161   let mutuals = 0;
162   for (const n of adj.get(id)) if (following.has(n)) mutuals += 1;
163
164   results.push({
165     id,
166     name: user.name,
167     role: user.role,
168     city: user.city,
169     interests: user.interests,
170     score: Number(score.toFixed(4)),
171     reasons: {
172       sharedInterests: shared,
173       mutualConnections: mutuals,
174       sameCity: cityScore(user, me) === 1,
175       complementaryRole: complementScore(me, user) ? user.role : null,
176       hasLinkedin: Boolean(user.linkedin),
177     },
178   });
179 }
180 results.sort((a, b) => b.score - a.score || a.id.localeCompare(b.id));
181 // match strength is relative to the best candidate (presentation, not probability)
182 const top = results[0]?.score || 1;
183 for (const r of results) r.matchPct = Math.round(99 * (r.score / top));
184 return results.slice(0, limit);
```

IN PLAIN LANGUAGE

Results carry shared interests, mutuals, city, role complement, and LinkedIn presence.

WHY THIS DECISION EXISTS

The match percentage is presented as relative strength, not a probability.

The recommendation engine in `server/engine.js` is deliberately a set of pure functions. It receives a graph snapshot and returns a ranking; it does not perform HTTP or database work.

Input graph

The graph contains:

- **users:** visible candidate profiles plus the requesting member.
- **follows:** directed follow relationships.

- **engagement:** bounded interaction histories between member pairs.

Supabase builds this graph in `loadGraphStore()`. SQLite builds the same shape in `loadGraphStore()` from `server/db.js`.

Signals

The direct profile/relationship weights sum to 1:

Signal	Weight	Meaning
Shared interests	0.30	Overlap in declared themes.
Mutual connections	0.20	Existing network proximity.
Engagement	0.20	Recent meaningful interactions.
Activity	0.10	Compatible contribution patterns.
City	0.10	Same normalized city label.
Complementary role	0.10	Disciplines likely to work well together.

The final ranking blends three perspectives:

- Graph traversal: 0.45
- Direct profile/relationship score: 0.30
- Collaborative filtering: 0.25

Graph traversal

`buildAdjacency()` converts directed follows into the adjacency model used for discovery.

`traversalScores()` walks up to three hops with a 0.5 decay for each additional hop. Nearby network relationships therefore matter more than distant ones.

Collaborative filtering

`cfScores()` asks a different question: which candidates are favored by members whose network behavior resembles mine? This adds network taste without relying only on profile text.

Engagement decay

`server/store.js` weights interaction types and applies a 30-day half-life. A recent message or follow matters more than an old passive view. Histories are bounded so repeated events cannot grow storage or scores without limit.

Final recommendation

`recommend()`:

- 1 Rejects an unknown source member.
- 2 Excludes the member and people already followed.
- 3 Scores each remaining candidate.
- 4 Applies a small LinkedIn verifiability prior when a validated profile URL exists.
- 5 Produces human-readable reasons such as shared interests, city, mutuals, or complementary role.
- 6 Converts raw values into relative match percentages.

The LinkedIn signal is only a validated URL today. NODAL does not currently call the LinkedIn API or perform LinkedIn OAuth.

Recommendation caching

`graphFingerprint()` hashes the relevant current graph state into the cache key. Results are stored for five minutes in `server/cache.js`.

- Local/default mode uses a bounded in-memory TTL cache.
- Production may use Redis through `REDIS_URL`.
- Production Redis must use TLS (`rediss://`) unless it is loopback.
- Cache failures degrade to misses instead of breaking the request.
- Follow and interaction writes invalidate affected recommendation entries.

The fingerprint means a changed consent/profile/graph snapshot naturally gets a different key. Explicit invalidation improves freshness after direct interactions.

14. External API Connections

Stripe Checkout is created on the server

SOURCE / `server/payments.js` / LINES 55-86

```
55 export async function createCheckoutSession({ cycle, origin, user }, config, fetchImpl =
56   fetch) {
57   const price = config.prices[cycle];
58   if (!price) throw Object.assign(new Error(`no price configured for ${cycle} billing`), {
59     status: 501 });
60   const body = new URLSearchParams({
61     mode: 'subscription',
62     client_reference_id: user.id,
63     'metadata[nodal_user_id]': user.id,
64     'subscription_data[metadata][nodal_user_id]': user.id,
65     'line_items[0][price]': price,
66     'line_items[0][quantity]': '1',
67     success_url: `${origin}/payments.html?checkout=success`,
68     cancel_url: `${origin}/payments.html?checkout=cancelled`,
69   });
70   if (user.email) body.set('customer_email', user.email);
71   const res = await fetchImpl(STRIPE_SESSIONS_URL, {
72     method: 'POST',
73     headers: {
74       Authorization: `Bearer ${config.secretKey}`,
75       'Content-Type': 'application/x-www-form-urlencoded',
76     },
77     body,
78   });
79   if (!res.ok) throw Object.assign(new Error('payment provider error'), { status: 502 });
80   const session = await res.json();
81   if (typeof session.url !== 'string') throw Object.assign(new Error('payment provider
82     error'), { status: 502 });
83   try {
84     const url = new URL(session.url);
85     if (url.protocol !== 'https:' || url.hostname !== 'checkout.stripe.com') throw new
86     Error('bad checkout URL');
87   } catch {
88     throw Object.assign(new Error('payment provider error'), { status: 502 });
89   }
90   return { url: session.url };
```

IN PLAIN LANGUAGE

The server selects a price, attaches NODAL metadata, provides trusted return URLs, and validates Stripe's URL.

WHY THIS DECISION EXISTS

The secret key and card form never enter NODAL's browser code.

A webhook is trusted only after cryptographic verification

SOURCE / server/payments.js / LINES 98-122

```

98 function safeEqualHex(a, b) {
99   const left = Buffer.from(a, 'hex');
100   const right = Buffer.from(b, 'hex');
101   return left.length === right.length && timingSafeEqual(left, right);
102 }
103
104 export function verifyStripeWebhook(payload, signatureHeader, secret, { now = Date.now } =
  {} ) {
105   if (!secret) throw Object.assign(new Error('webhook secret is not configured'), { status:
    503 });
106   const { timestamp, signatures } = parseStripeSignature(signatureHeader);
107   const ts = Number(timestamp);
108   if (!Number.isFinite(ts) || !signatures.length) {
109     throw Object.assign(new Error('invalid webhook signature'), { status: 400 });
110   }
111   const age = Math.abs(Math.floor(now() / 1000) - ts);
112   if (age > WEBHOOK_TOLERANCE_SEC) throw Object.assign(new Error('stale webhook signature'),
    { status: 400 });
113   const expected = createHmac('sha256',
    secret).update(`${timestamp}.${payload}`).digest('hex');
114   if (!signatures.some((sig) => safeEqualHex(sig, expected))) {
115     throw Object.assign(new Error('invalid webhook signature'), { status: 400 });
116   }
117   try {
118     return JSON.parse(payload);
119   } catch {
120     throw Object.assign(new Error('invalid webhook payload'), { status: 400 });
121   }
122 }

```

IN PLAIN LANGUAGE

The signature is compared with an HMAC over timestamp plus raw payload; stale or malformed messages fail.

WHY THIS DECISION EXISTS

A request that merely looks like Stripe is not sufficient proof.

14.1 Supabase Auth API

NODAL calls Supabase /auth/v1 endpoints from server/supabase.js for:

- Sign-up.
- Password grant login.
- Current Auth user validation.
- Refresh-token exchange.
- Logout.
- Admin account deletion.

The member's browser calls only NODAL routes. This gives NODAL one place to apply rate limits, normalize errors, set cookies, and initialize application records.

14.2 Supabase PostgREST API

The same module calls /rest/v1 for tables and /rest/v1/rpc/apply_stripe_event for the atomic billing function. Structured query parameters and JSON bodies are used instead of building SQL in application code.

14.3 Stripe API

server/payments.js posts form-encoded data to Stripe's Checkout Sessions endpoint.

The server supplies:

- The configured Stripe price ID.

- The authenticated member's email and NODAL user ID.
- A success URL and cancel URL derived from trusted deployment configuration.
- The membership billing cycle selected by the member.

The browser receives only the Stripe-hosted checkout URL. Stripe collects card details, so NODAL never stores card numbers, CVCs, or full payment data.

Webhook processing takes the exact raw request bytes and checks Stripe's HMAC signature with a timing-safe comparison. The timestamp must be within five minutes. Only then does the code parse and apply the event.

14.4 GeoDB Cities API

`createCitySearch()` proxies the public GeoDB city service. The frontend sends a short query to `/api/cities`; the server:

- Requires an authenticated member.
- Normalizes and length-limits the query.
- Encodes provider parameters safely.
- Paces outbound calls.
- Caches results for 24 hours.
- Returns a stable NODAL response shape.
- Returns an empty list if the provider is temporarily unavailable.

This avoids provider details being spread through the UI and gives us one place to replace GeoDB later.

14.5 Redis

Redis is optional and used only as a cache. The app implements the small command surface it needs and enforces connection, response-size, timeout, authentication, database, and TLS rules. If Redis is unavailable, the application still computes recommendations from the source data.

14.6 Integrations not active yet

- LinkedIn OAuth is not active; only validated LinkedIn profile URLs are supported.
- Stripe code is ready, but checkout remains unavailable unless complete server-side credentials and price IDs are configured.
- Supabase email delivery and rate limits are operational provider settings. The code sanitizes those failures, but SMTP capacity is configured in Supabase, not in this repository.

15. Supabase Database Schema

The migrations are append-only and ordered. They should be read chronologically.

20260709_production_core.sql

Creates:

- `profiles`
- `profile_preferences`
- `onboarding_responses`
- `organizations`
- `organization_memberships`
- `stripe_customers`
- `member_follows`
- `member_interactions`
- `stripe_events`

It also creates indexes, `updated_at` triggers, RLS policies, the public-profile view, and the Stripe event function.

The profile ID is the Supabase Auth user ID. Foreign keys use `on delete cascade`, so deleting the Auth user removes user-owned application records.

20260710000000_security_hardening.sql

Hardens the directory projection and replaces the Stripe mutation with an event-aware atomic implementation. The function records event identity and applies only a valid newer state transition.

20260710010000_advisor_hardening.sql

- Makes the profile view `security_invoker`.
- Removes direct `anon` and `authenticated` access to the view.
- Grants directory projection access only to the server-side service role.
- Adds an explicit client-deny policy for the Stripe event ledger.

20260710020000_rls_performance.sql

Adds a follow-target index and rewrites ownership policies in a form that performs better under Supabase's RLS planner.

16. How Row Level Security Works

A profile row belongs to its Auth user

SOURCE / supabase/migrations/20260709_production_core.sql / LINES 160-178

```
160 alter table public.profiles enable row level security;
161 alter table public.profile_preferences enable row level security;
162 alter table public.onboarding_responses enable row level security;
163 alter table public.organizations enable row level security;
164 alter table public.organization_memberships enable row level security;
165 alter table public.stripe_customers enable row level security;
166 alter table public.member_follows enable row level security;
167 alter table public.member_interactions enable row level security;
168 alter table public.stripe_events enable row level security;
169
170 drop policy if exists "profiles_select_own" on public.profiles;
171 create policy "profiles_select_own"
172 on public.profiles for select to authenticated
173 using (auth.uid() = id);
174
175 drop policy if exists "profiles_insert_own" on public.profiles;
176 drop policy if exists "profiles_update_own" on public.profiles;
177
178 drop policy if exists "profile_preferences_select_own" on public.profile_preferences;
```

IN PLAIN LANGUAGE

RLS is enabled and the row ID is compared with Supabase's verified `auth.uid()`.

WHY THIS DECISION EXISTS

Authentication alone does not grant access to every member's data.

The directory view became server-led

SOURCE / supabase/migrations/20260710010000_advisor_hardening.sql / LINES 4-29

```
4 create or replace view public.public_profiles
5 with (security_invoker = true) as
6 select
7   p.id,
8   p.preferred_name,
9   p.full_name,
10  p.avatar_url,
11  p.city_region,
12  p.country,
13  p.languages,
14  p.affiliation,
15  p.bio,
16  p.public_role,
17  p.created_at,
18  p.updated_at
19 from public.profiles p
20 join public.profile_preferences pref on pref.user_id = p.id
21 where coalesce(pref.data_consent -> 'directoryPublic', 'false'::jsonb) = 'true'::jsonb;
22
23 revoke all on public.public_profiles from anon, authenticated;
24 grant select on public.public_profiles to service_role;
25
26 drop policy if exists "stripe_events_deny_client" on public.stripe_events;
27 create policy "stripe_events_deny_client"
28 on public.stripe_events for all to anon, authenticated
29 using (false)
```

IN PLAIN LANGUAGE

Direct anonymous and member access is revoked; the trusted service role receives the projection.

WHY THIS DECISION EXISTS

The server controls consent-aware output and keeps the Stripe event ledger private.

RLS is enabled for every user-owned or sensitive table.

The policy model is:

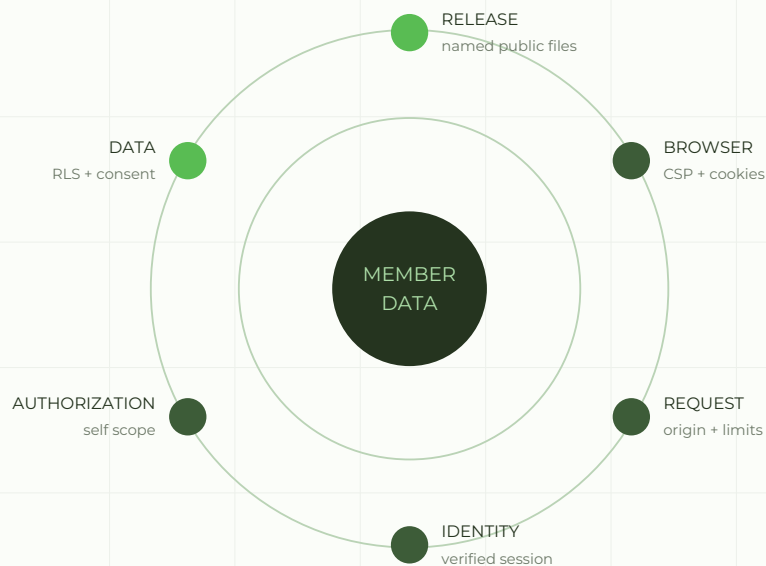
- A user may read their own profile, preferences, onboarding responses, and Stripe metadata.
- A user may read only organization memberships relevant to them.
- Follow rows are visible only to participants.

- Interaction rows are visible only to their owner under the defined policy.
- Stripe event ledger rows are denied to ordinary clients.
- The restricted directory projection is consumed by the trusted server, not queried directly by arbitrary browsers.

NODAL's server often uses the Supabase secret/service role, which can bypass RLS. Therefore RLS is defense in depth, not a replacement for route authorization. The router must still derive the acting user from the session and must never trust a user ID supplied by the browser. `resolveUserId()` and the `/api/me` design enforce that rule.

17. The Security Model

SECURITY ORBITS THE MEMBER DATA, LAYER BY LAYER



Response headers constrain the browser

SOURCE / server/server.js / LINES 74-115

```
74 function securityHeaders(headers = {}) {
75   const h = {
76     'X-Content-Type-Options': 'nosniff',
77     'X-Frame-Options': 'DENY',
78     'X-Permitted-Cross-Domain-Policies': 'none',
79     'Referrer-Policy': 'strict-origin-when-cross-origin',
80     'Permissions-Policy': 'camera=(), microphone=(), geolocation=(), payment=()',
81     'Cross-Origin-Opener-Policy': 'same-origin',
82     'Cross-Origin-Resource-Policy': 'same-origin',
83     'Origin-Agent-Cluster': '?1',
84     ...headers,
85   };
86   if (process.env.NODE_ENV === 'production') {
87     h['Strict-Transport-Security'] = 'max-age=31536000; includeSubDomains';
88   }
89   return h;
90 }
91
92 function contentSecurityPolicy(env = process.env) {
93   const directives = [...BASE_CSP];
94   if (env.NODE_ENV === 'production') directives.push('upgrade-insecure-requests');
95   return directives.join('; ');
96 }
97
98 function htmlSecurityHeaders(headers = {}) {
99   return securityHeaders({ 'Content-Security-Policy': contentSecurityPolicy(), ...headers
100 });
101 }
102
103 function safeNext(value) {
104   if (typeof value !== 'string' || !value.startsWith('/') || value.startsWith('/')) return
105     '/dashboard.html';
106   if (value.includes('\\') || /[\\u0000-u001f\u007f]/.test(value)) return '/dashboard.html';
107   try {
108     const parsed = new URL(value, 'https://nodal.local');
109     if (parsed.origin !== 'https://nodal.local') return '/dashboard.html';
110     const normalized = path.posix.normalize(parsed.pathname);
111     if (!normalized.startsWith('/') || normalized.startsWith('/')) return
112       '/dashboard.html';
113     return `${normalized}${parsed.search}${parsed.hash}`;
114   } catch {
115     return '/dashboard.html';
116   }
117 }
```

IN PLAIN LANGUAGE

Responses receive MIME, frame, referrer, permission, origin, transport, CSP, and redirect controls.

WHY THIS DECISION EXISTS

These limits reduce clickjacking, injection impact, leakage, and redirect abuse.

Mutations require same-origin and bounded JSON

SOURCE / server/server.js / LINES 347-377

```

347 function sameOrigin(req) {
348   const site = req.headers['sec-fetch-site'];
349   if (site && site !== 'same-origin' && site !== 'none') return false;
350   const origin = req.headers.origin;
351   if (origin) {
352     try { return new URL(origin).host === req.headers.host; } catch { return false; }
353   }
354   return true;
355 }
356
357 function readJsonBody(req) {
358   return new Promise((resolve, reject) => {
359     if (!/^application\/json/.test(req.headers['content-type'] ?? '')) {
360       reject(Object.assign(new Error('expected application/json'), { status: 415 }));
361       return;
362     }
363     let size = 0;
364     const chunks = [];
365     req.on('data', (c) => {
366       size += c.length;
367       if (size > MAX_BODY) { req.destroy(); reject(Object.assign(new Error('body too
large'), { status: 413 })); return; }
368       chunks.push(c);
369     });
370     req.on('end', () => {
371       try { resolve(JSON.parse(Buffer.concat(chunks).toString() || '{}')); }
372       catch { reject(Object.assign(new Error('invalid JSON'), { status: 400 })); }
373     });
374     req.on('error', reject);
375   });
376 }
377

```

IN PLAIN LANGUAGE

Origin is checked, JSON content type is required, and request bodies stop at 32 KB.

WHY THIS DECISION EXISTS

Another website cannot casually reuse a member cookie, and oversized input fails early.

Security is layered because no single control is sufficient.

Threat: a secret appears in Git or browser code

Controls:

- Real `.env*`, `.vercel/`, local databases, Supabase temp state, and generated output are ignored.
- `.env.example` contains names and safe placeholders only.
- The browser calls NODAL, not Supabase admin or Stripe directly.
- The production build copies an explicit frontend allowlist.
- Runtime configuration distinguishes public and server-only variables and rejects swapped Supabase key classes.

Threat: an anonymous visitor opens private UI or APIs

Controls:

- Private HTML routes are checked before the file is served.
- Protected APIs call `requireAuth()`.
- Login redirects use safe local destinations.
- The frontend's redirect is convenience; the server check is authoritative.

Threat: one member reads or changes another member's data

Controls:

- Routes use `me` or the authenticated session UUID.
- `resolveUserId()` rejects a mismatched requested user.
- Profile updates always use `sessionUser.id`.
- Supabase RLS restricts ordinary database access.
- The directory includes only members who granted directory consent.

Threat: a cross-site page submits a mutation using the member's cookies

Controls:

- Session cookies are `SameSite=Lax`.
- Every state-changing browser route checks `Origin/same-origin` with `sameOrigin()`.
- Cross-origin login, signup, logout, profile writes, follow events, interactions, deletion, and checkout are rejected.

Threat: session theft through browser JavaScript

Controls:

- Access, refresh, and local session cookies are `HttpOnly`.
- Cookies are `Secure` in production.
- Tokens are not put in URLs or `localStorage`.
- SQLite stores a hash of the session token, not the raw token.
- Logout revokes the provider session where possible and clears cookies.

Threat: malicious or oversized input

Controls:

- JSON content type is required for JSON mutations.
- JSON and webhook body sizes are capped.
- Email, password, ID, enum, city query, URL, text length, array length, and profile-field rules are validated.
- Unexpected nested or server-owned profile state is normalized or ignored.
- Error responses do not return database/provider internals for server failures.

Threat: brute-force login or recommendation poisoning

Controls:

- Authentication attempts are rate limited before expensive password work.
- Interaction writes are rate limited per authenticated user.
- Allowed interaction types are fixed.
- Pair histories are bounded.
- A member cannot target themselves or a nonexistent user.
- The target member's own recommendation deck is not built from untrusted inbound behavior as though they initiated it.

Threat: forged or replayed Stripe events

Controls:

- Webhook secret stays server-only.
- HMAC is checked against the exact raw body.
- Signature comparison is timing-safe.
- Timestamps older than the tolerance are rejected.
- Event IDs make retries idempotent.
- Event creation time, event rank, and event ID provide deterministic ordering.
- One database RPC records and applies an event atomically.
- Card data is never stored by NODAL.

Threat: accidental source or operational-file exposure

Controls:

- Static delivery is allowlist-based.
- Requests for `server/`, `scripts/`, `tests/`, `web/`, `docs/`, `supabase/`, package metadata, dotfiles, or deployment metadata return `404`.
- Vercel packages only the runtime modules and protected page sources needed by the function.
- Production assets are generated from named source files.

Browser hardening

The server applies:

- Content Security Policy.
- `X-Content-Type-Options: nosniff`.
- `X-Frame-Options: DENY`.
- Strict referrer policy.
- Restrictive Permissions Policy.
- Cross-origin opener/resource protections.
- HSTS in production.

These controls reduce script injection impact, clickjacking, MIME confusion, unnecessary browser capability access, and cross-origin data leakage.

18. Important Design Decisions and Their Tradeoffs

Keep the existing frontend stack

Decision: Preserve plain HTML/CSS/JS.

Why: The interface already existed, the immediate need was production behavior and security, and a framework rewrite would have mixed visual regressions with backend risk.

Tradeoff: Less tooling and less bundle weight, but larger controller files require discipline as features grow.

Put a server boundary in front of Supabase

Decision: Browser code calls NODAL API routes rather than directly performing application data writes against Supabase.

Why: Centralized validation, cookie handling, rate limiting, stable response models, protected service credentials, and one policy path for Supabase and SQLite.

Tradeoff: More server code and an extra network hop, but a much smaller trusted browser surface.

Use Supabase in production and SQLite locally

Decision: One repository contract, two persistence adapters.

Why: Supabase provides durable Postgres, managed Auth, email confirmation, and RLS. SQLite gives fast, self-contained local development and tests.

Tradeoff: Both adapters must stay behaviorally aligned, which is why repository contract tests are important.

Serve assets statically but HTML through Node

Decision: JS/CSS/WebP go to Vercel static delivery; HTML remains behind the app.

Why: Assets become fast and cacheable while private pages still receive server-side authorization checks.

Tradeoff: The build needs an explicit source-to-public mapping.

Make recommendations pure and data-source independent

Decision: Compute rankings from a graph snapshot.

Why: Deterministic tests, explainable weights, database independence, and straightforward caching.

Tradeoff: The repository currently loads a full visible graph for a recommendation request. That is appropriate for the present network size but will need query/precomputation changes at much larger scale.

Keep payments fail-closed

Decision: Show a clear unavailable/preview state until every required Stripe setting exists.

Why: Partial payment configuration is more dangerous than no payment configuration.

Tradeoff: Billing cannot be "half tested" in production; the complete credential and webhook setup must be finished together.

Use an explicit deployment allowlist

Decision: Name every public file instead of trying to block sensitive patterns after copying everything.

Why: New internal files remain private by default.

Tradeoff: A newly added public asset must also be added to the build and static maps.

19. Reliability and Performance Work

The in-memory cache is expiring and bounded

SOURCE / server/cache.js / LINES 9-35

```

 9 export class MemoryCache {
10   constructor({ now = Date.now, maxEntries = 1000 } = {}) {
11     this.now = now;
12     this.maxEntries = Math.max(1, Number(maxEntries) || 1000);
13     this.store = new Map();
14   }
15   async get(key) {
16     const entry = this.store.get(key);
17     if (!entry) return null;
18     if (entry.expires <= this.now()) { this.store.delete(key); return null; }
19     return entry.value;
20   }
21   async set(key, value, ttlMs) {
22     const now = this.now();
23     for (const [storedKey, entry] of this.store) {
24       if (entry.expires <= now) this.store.delete(storedKey);
25     }
26     this.store.delete(key);
27     this.store.set(key, { value, expires: this.now() + ttlMs });
28     while (this.store.size > this.maxEntries) {
29       this.store.delete(this.store.keys().next().value);
30     }
31   }
32   async del(...keys) {
33     for (const key of keys) this.store.delete(key);
34   }
35 }

```

IN PLAIN LANGUAGE

Entries have expiry times; stale values are removed; the oldest entries leave when capacity is reached.

WHY THIS DECISION EXISTS

A speed feature cannot grow memory forever and can be replaced by Redis through the same interface.

The production work was not only about features. Several changes address failure modes seen during deployment:

- Email-confirmation signup responses are handled without assuming an immediate session.
- Repeated confirmed-email signup is translated into a useful sign-in path.
- Supabase email rate-limit and database errors are replaced with safe user-facing messages.
- Missing profile preferences can be repaired without overwriting the profile.
- Expired Supabase access tokens refresh through the HttpOnly refresh cookie.
- Static assets bypass session resolution.
- Static files receive cache headers.
- PNG artwork was converted to production WebP assets.
- Source PNGs stay outside the production bundle.
- GeoDB responses and recommendation snapshots are cached.
- Vercel preview URLs are accepted only through trusted environment fallback rules.

- Production configuration validation stops broken deploys early.

20. Tests as Executable Documentation

A security statement becomes an executable test

SOURCE / tests/api.test.js / LINES 135-143

```
135 test('cross-origin POSTs are rejected', async (t) => {
136   const base = await boot(t);
137   const r = await fetch(`${base}/api/users/you/follow`, {
138     method: 'POST',
139     headers: { 'Content-Type': 'application/json', Origin: 'https://evil.example' },
140     body: JSON.stringify({ targetId: 'flavia' }),
141   });
142   assert.equal(r.status, 403);
143 });
```

IN PLAIN LANGUAGE

The test submits a mutation from a hostile Origin and expects HTTP 403 plus the public error.

WHY THIS DECISION EXISTS

A future refactor that removes the guard fails the build instead of silently weakening production.

Run:

```
npm test
```

The 77 tests are divided by responsibility:

- tests/engine.test.js: recommendation weights, exclusions, graph depth, decay, reasons, city, role complement, and LinkedIn signal.
- tests/cache.test.js: TTL, eviction, invalidation, Redis TLS/auth/database settings, timeouts, and oversized replies.
- tests/api.test.js: auth, profile persistence, privacy, CSRF, rate limits, recommendations, Stripe, city proxy, protected pages, path traversal, and static caching.
- tests/supabase.test.js: environment key separation, Auth response variants, session refresh, profile repair/update, mentor policy, and atomic Stripe RPC use.
- tests/deploy.test.js: Vercel static/function routing, repository organization, migration/RLS safety, CI pinning, and release-tree exclusions.

The build command is itself a gate:

```
npm run build
```

It runs all tests before generating public/. A failing behavioral or security test therefore prevents a normal production build.

21. What Changed Over the Life of the Repository

The Git history shows these broad phases:

- ¹ Initial interface: the new NODAL website and early interactions were created.
- ² Visual/product expansion: dashboard, profile, Sistema Urbano content, tabs, force graph, matching surfaces, and multilingual copy evolved.
- ³ Recommendation expansion: city, complementary-role, engagement recency, graph traversal, collaborative signals, and LinkedIn verifiability were added.
- ⁴ Payment architecture: Stripe checkout, provider separation, preview fallback, and trusted return URLs were introduced.
- ⁵ Real identity: the dashboard and public profile were bound to the active member rather than a hardcoded identity.
- ⁶ Production conversion: server-backed auth, persistence, Supabase schema/RLS, privacy actions, and deployment validation replaced unsafe mock assumptions.
- ⁷ Deployment correction: Vercel function packaging, root routing, assets, Supabase policy issues, and Node runtime configuration were corrected.
- ⁸ Auth reliability: confirmed-email responses, repeated signup, and provider email failures were handled safely.
- ⁹ Performance: static delivery, reduced session work, cache headers, and optimized images improved loading.
- ¹⁰ Repository hygiene: tests, operational scripts, frontend sources, editable images, generated output, and architecture docs were separated by responsibility.
- ¹¹ Security/history hygiene: secrets and internal tooling artifacts were excluded, release boundaries were hardened, and public history was cleaned before publication.

The result is not a new application pasted over the old one. It is the original design progressively surrounded by real data, server ownership, deployment discipline, tests, and security controls.

22. Current Limitations and Future Work

These are important to understand while studying the code:

- LinkedIn OAuth is prepared conceptually but not implemented.
- Stripe requires real price, secret, webhook, and deployment URL configuration before live billing.
- Supabase confirmation email capacity depends on the project's SMTP/provider setup.
- Some dashboard content categories remain static editorial examples.
- The in-memory rate limiter is per application instance. A distributed production limit would require Redis-backed limiting or Vercel Firewall/rate-limit rules.
- The Supabase admin client bypasses RLS, so every future server route must continue deriving ownership from the authenticated session.
- The recommendation graph is assembled in application memory. It is clear and testable at current scale, but a much larger network may need SQL-side candidate generation or offline precomputation.
- `dashboard.js` is large and should eventually be split by feature without changing the public script contract.
- There is no general admin console or moderation workflow yet.

These are boundaries, not hidden claims. The current code is strongest when it is explicit about what is real, what is provider-configured, and what is only prepared for later.

23. Recommended Reading Order

Pass 1: understand the shape

- 1 `README.md`
- 2 `docs/architecture/repository-organization.md`
- 3 `vercel.json`
- 4 `scripts/build-static.js`
- 5 `api/index.js`

Pass 2: follow one authenticated request

- 1 `web/scripts/auth.js`
- 2 `server/server.js` from `createApp()`
- 3 `server/repository.js`
- 4 `server/supabase.js` from `resolveSession()`, `signup()`, and `login()`
- 5 `supabase/migrations/20260709_production_core.sql`

Pass 3: follow profile state

- 1 `web/scripts/dashboard.js`: `normalizeApiUser()`, `serializeUser()`, `saveProfile()`
- 2 `server/server.js`: `sanitizeProfilePatch()` and `PATCH /api/me`
- 3 `server/profile-policy.js`
- 4 `server/supabase.js`: `bundleForUser()` and `upsertProfileState()`
- 5 `tests/supabase.test.js` profile tests

Pass 4: follow recommendations

- 1 `web/scripts/recs.js`
- 2 `server/server.js`: `recommendation`, `follow`, and `interaction` routes
- 3 `server/supabase.js`: `loadGraphStore()`
- 4 `server/store.js`
- 5 `server/engine.js`
- 6 `tests/engine.test.js`

Pass 5: study the security story

- 1 `server/server.js`: `headers`, `CSP`, `path normalization`, `same-origin`, `body readers`, `auth guards`
- 2 `server/auth.js`
- 3 `server/payments.js`
- 4 All four Supabase migrations in order
- 5 Security-focused cases in `tests/api.test.js`, `tests/supabase.test.js`, and `tests/deploy.test.js`

24. Practical Study Exercises

- 1 Trace `POST /api/auth/login` from the form submission to the Supabase token response and back to the Set-Cookie header.
- 2 Trace one profile field, such as `city`, from the dashboard input through `serializeUser()`, `sanitizeProfilePatch()`, and `upsertProfileState()` into `profiles.city_region`.
- 3 Explain why `mentorApplied` cannot simply be trusted from the browser payload.
- 4 Build a three-user graph on paper and calculate how one follow changes traversal and collaborative scores.
- 5 Trace a `customer.subscription.updated` webhook from HMAC verification to `apply_stripe_event`.
- 6 Try to identify every point that prevents a user from editing another user's profile.
- 7 Compare `server/supabase.js` and the SQLite repository and list the shared contract operations.
- 8 Add a hypothetical new public image on paper and identify every build/static allowlist that would need to include it.
- 9 Read one security test before its implementation and predict the control it expects.
- 10 Draw the difference between authentication, authorization, consent filtering, RLS, and browser hardening. NODAL uses all five for different purposes.

25. The Core Mental Model

When reading any NODAL feature, ask five questions:

- 1 What does the browser render? Look in `web/`.
- 2 Which NODAL API owns the action? Look in `createApp()`.
- 3 How is the acting member established? Follow session resolution and ownership checks.
- 4 Where is the durable state stored? Follow the repository into Supabase or SQLite.
- 5 Which failure or threat is handled? Look for validation, policy, rate limit, cache behavior, error translation, and tests.

That pattern explains nearly the entire repository. The frontend is intentionally thin around trusted decisions; the server turns a session into an authorized action; the repository translates that action into durable storage; and tests preserve the contract across local and production backends.

FINAL MENTAL MODEL

The browser expresses intent. The server proves identity and applies policy. The repository changes durable state. Tests preserve the contract.